



MINIZINC

MODELITZACIÓ DE CSPS/COPS: ALGUNES ORIENTACIONS PER MINIZINC

- ▶ Modelatge amb MiniZinc
 - ▶ Predicats:
 - ▶ Limitacions globals
 - ▶ Restriccions i proves definides per l'usuari
 - ▶ Funcions de reflexió
 - ▶ Variables locals
 - ▶ Funcions de negació i parcials
- ▶ Eficiència
- ▶ Diferents models de problemes
- ▶ Restriccions redundants

PREDICATS

- ▶ MiniZinc ens permet capturar restriccions complexes en un predicat.
- ▶ El predicat pot ser
 - ▶ Amb el suport del solucionador subjacent, o
 - ▶ Definit per l'usuari
- ▶ Una definició de predicat té la forma
 - ▶ `predicate <nom_predicat>(<arg-def> ... <arg-def>)=<expressió-bool>`
- ▶ Una definició d'argument (<arg-def>) és una declaració de tipus MiniZinc,
- ▶ Exemple: jobshop

PREDICATS: EXEMPLE: JOBSHOP

```
predicate no_overlap(var int:s1, int:d1, var int:s2,  
int:d2) = s1 + d1 <= s2 \/ s2 + d2 <= s1;
```

- ▶ Això defineix una nova restricció que obliga a que una tasca amb hora d'inici $s1$ i durada $d1$ no se superposi amb una tasca amb hora d'inici $s2$ i durada $d2$. Ara es pot utilitzar dins del model en qualsevol lloc on es pugui utilitzar qualsevol altra expressió booleana (que inclogui variables de decisió).
- ▶ Exemple jobshop2

RESTRICCIONS GLOBALS: TOTS DIFERENTS

- `all_different(array[int] of var int:x)`
- ▶ Defineix un subproblema d'assignació: totes les vars d'`x` prenen un valor diferent
- ▶ Per utilitzar `-lo`, hem d'incloure-lo o incloure tots els globals amb `include "globals.mzn"`

```
include "all_different.mzn";
var 1..9: S;
var 0..9: E;
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;
constraint 1000 * S + 100 * E + 10 * N + D
+ 1000 * M + 100 * O + 10 * R + E
= 10000 * M + 1000 * O + 100 * N + 10 * E + Y;
constraint all_different([S,E,N,D,M,O,R,Y]);
solve satisfy;
```

RESTRICCIONS GLOBALS: INVERSA

`inverse(array[int] of var int:f, array[int] of var int:if)`

$f[i] = j \Leftrightarrow if[j] = i$ (if és la funció inversa f^{-1})

- ▶ Útil per a problemes d'assignació on volem ambdues vistes del problema
 - ▶ `array[1..n] of var 1..n: t %tasca;`
 - ▶ `array[1..n] of var 1..n: tr %treballador;`
 - ▶ `constraint inverse(t,tr);`
- ▶ Podem expressar limitacions sobre tasques i treballadors, p.ex:
`constraint t[1] > t[2] /\ tr[3] < tr[4];`
- ▶ la tasca del treballador 1 té una numeració posterior a de la tasca del treballador 2
- ▶ el treballador de la tasca 3 rep una numeració anterior a la del treballador de la tasca 4

RESTRICCIONS GLOBALS: ACUMULATIVA

- ▶ La restricció acumulativa s'utilitza per descriure l'ús acumulat de recursos.
`cumulative(array[int] of var int: s, array[int] of var int: d, array[int] of var int: r, var int: b)`

Obliga a que un conjunt de tasques donades pels temps d'inici s , les durades d i els requisits de recursos r , mai requereixin més d'un recurs global b en cap moment.

- ▶ Exemple: moure mobles

RESTRICCIONS GLOBALS: TABLE

La restricció de table fa que una tupla de variables prengui un valor d'un conjunt de tuples. Com que no hi ha tuples a MiniZinc, això es codifica mitjançant matrius. L'ús de la taula té una de les dues formes:

```
table(array[int] of var bool: x, array[int, int] of bool: t)  
table(array[int] of var int: x, array[int, int] of int: t)
```

depenent de si les tuples són booleans o enteres.

La restricció obliga a $x \in t$ on considerem que x i cada fila de t és una tupla, i t un conjunt de tuples.

Exemple: dinar equilibrat

▶

RESTRICCIONS DEFINIDES PER L'USUARI

Minizinc permet definir restriccions per part de l'usuari

- predicates (var bool)
- tests (bool)

```
int: n;  
n=8;  
array [1..n] of var 1..n: q;  
predicate  
noattack(int: i, int: j, var int: qi, var int: qj) =  
qi != qj /\ qi + i != qj + j /\ qi - i != qj - j;  
constraint  
forall (i in 1..n, j in i+1..n) (noattack(i, j, q[i], q[j]));  
solve satisfy;  
  
output [ if fix(q[j]) == i then "q" else "." endif ++  
if j == n then "\n" else "" endif | i,j in 1..n];
```

REFLECTIONS

Per ajudar a escriure tests i predicats genèrics, hi ha diverses funcions de reflexió retornen informació sobre conjunts d'índexos d'arrays, dominis de variables que són conjunts (sets) i rangs de variables. Els dels conjunts d'índexs són

`index_set(<1-D array>)`, `index_set_1of2(<2-D array>)`, `index_set_2of2(<2-D array>`

Exemple: jobshop (jobshop3.mzn)

Un model millor del jobshop reuneix totes les restriccions de no solapament per a una única màquina en una única **restricció disjuntiva**.

```
constraint %% ensure no overlap of tasks
  forall(j in TASK) (
    disjunctive([s[i,j] | i in JOB], [d[i,j] | i in JOB])
  );
```

REFLECTIONS:ALTRES

```
dom(<arith-dec-var>)  
lb(<arith-dec-var>)  
ub(<arith-dec-var>)  
lb_array(<var-set>)  
ub_array(<var-set>)
```

Aquesta darrera classe ofereix "aproximacions segures" al domini inferit, límit inferior i superior

VARIABLES LOCALS

Sovint és útil introduir variables locals en un test o predicat

L'expressió **let** ens permet fer-ho

let { <var-dec>, ... } **in** <exp>

(També es pot utilitzar en altres expressions)

La declaració **var** pot contenir variables i paràmetres de decisió

– Els paràmetres s'han d'inicialitzar

Exemple:

let {int: l = lb(x), int: u = ub(x) div 2, var l .. u: y} en x = 2*y

INTERPRETACIÓ DE LA NEGACIÓ

Les variables locals no poden aparèixer en un context negat

```
not <bool-exp>  
<bool-exp> -> <bool-exp>  
<bool-exp> = <bool-exp>
```

Això és perquè no tindran el significat correcte

```
predicate even(var int:x) = let {var int:y } in x = 2*y;  constraint not even(z);
```

Es traduiria a (més informació sobre la traducció més endavant)

```
let { var int:y } in not (z = 2 * y)
```

Solution: $z = 2$, $y = 0$!

Solvers no donen suport a

– forall y. not (z = 2 * y)

EFICIÈNCIA

- ▶ A més de modelar correctament el nostre problema també volem que el nostre model MiniZinc es resolgui de manera eficient
- ▶ La informació sobre l'eficiència s'obté mitjançant les flags de MiniZinc
 - ▶ estadístiques del *solver* [nombre de punts d'elecció]
 - ▶ estadístiques [nombre de punts d'elecció, memòria i ús del temps]
- ▶ Es requereix una àmplia experimentació per determinar l'eficiència relativa
- ▶ Per tal de fer-ho podem:
- ▶ Afegir anotacions de cerca a l'element de resolució per controlar l'exploració de l'espai de cerca.
- ▶ Utilitzeu restriccions globals com ara `all_different`, ja que tenen un millor comportament de propagació.
- ▶ Proveu diferents models per al problema.
- ▶ Afegiu restriccions redundants.
- ▶ Tingueu cura d'evitar algunes solucions problemàtiques simples:
 - ▶ Variables lligades tan estretament com sigui possible
 - ▶ Eviteu `var int` si és possible
- ▶ Eviteu introduir variables innecessàries
- ▶ Feu iteracions el més acurades possible

EFICIÈNCIA: LIMITAR VARIABLES

```
var int: x;  var int: y;  
constraint x <= y /\ x > y;  
solve satisfy;
```

```
var -1000..1000: x;  
var -1000..1000: y;  
constraint x <= y /\ x > y;  
solve satisfy;
```

Compareu el temps que triga a indicar que és insatisfactible

EFICIÈNCIA: ANOTACIONS CERCA

- ▶ MiniZinc permet el control de la cerca mitjançant anotacions per al *solver*
- ▶ Per a variables enteres aquestes tenen forma
- ▶ `int_search(vars, var_select, choice, explore)`
- ▶ `vars` és un array d'una dimensió que especifica les variables var int afectades per l'anotació;
- ▶ `var_select` és l'estratègia de selecció de variables
 - ▶ `smallest`, `largest`, `occurrence`,
`most_constrained`,
- ▶ `choice` és l'estratègia d'elecció de valor `indomain`, `indomain_min`, `indomain_max`, `indomain_split`,...
- ▶ `explore` és l'estratègia de cerca
 - ▶ `complete`, `bbs(s)`, `fail`,...
 - ▶

EFICIÈNCIA: ANOTACIONS CERCA: PARAMETRES

- `indomain_min`: assigna a la variable el valor de domini més petit,
- `indomain_median`: assigna a la variable el valor mitjà de domini (o el més petit dels dos valors mitjans en cas que hi hagi un nombre parell d'elements al domini),
- `indomain_random`: assigna a la variable un valor aleatori del seu domini,
- `indomain_split`: parteix el domini de les variables excloent la meitat superior.
- `input_order`: selecciona en el mateix ordre de l'array
- `first_fail`: escull la variable amb el tamany de domini més petit
- `smallest`: escull la variable que tingui el valor més petit en el seu domini

EFICIÈNCIA: ANOTACIONS CERCA: N-REINES

► Proveu:

```
int: n;
n=8;
array [1..n] of var 1..n: q;
predicate
noattack(int: i, int: j, var int: qi, var int: qj) =
qi != qj /\ qi + i != qj + j /\ qi - i != qj - j;
constraint
forall (i in 1..n, j in i+1..n) (noattack(i, j, q[i], q[j]));
solve ::
int_search(q,first_fail,indomain_min,complete)
satisfy;
  output [ if fix(q[j]) == i then "Q" else "." endif ++
if j == n then "\n" else "" endif | i,j in 1..n];
```

RESOLDRE: SUBHASTA COMBINATÒRIA

Si tenim aquestes ofertes d'una subhasta combinatòria:

$$\begin{array}{ll} o1 = \{1,2,3,4\}, & r1 = 8 \\ o2 = \{2,3,6\}, & r2 = 6 \\ o3 = \{1,4,5\}, & r3 = 5 \\ o4 = \{2,8\}, & r4 = 2 \\ o5 = \{5,6\}, & r5 = 2 \end{array}$$

Quina és l'assignació òptima?